

spec

Feature Specification: Chapter Hierarchy

Feature Branch: 003-chapter-hierarchy

Created: 2026-09-05

Status: Draft

Input: No operator directive is quoted here, because none was given. This feature was opened by a **change in the tree**, not by a request, and saying so plainly is the honest form: on 2026-09-05, commit 266f443 added workshop/chapters/02 and workshop/chapters/02.01 — **the first sub-chapter this project has ever had**. Everything downstream of chapters/ was written against a flat list of chapters, and three defects that were latent with one chapter went live with three. That commit fixed three of them. This specification is the rest.

Why this feature exists: one directory name broke an assumption nobody had written down

Nothing in the platform ever *decided* that chapters were flat. It simply never had a reason not to be. With one chapter, 01, every shortcut worked: parse the leading digits and you have an ordinal; match chapter-\d+ and you have every chapter; compare a scope with HasSuffix and you have the right chapter. Each of those is correct for 01 and each is wrong for 02.01.

The three that were already found share a shape worth naming before the requirements, because it is what this specification is organised around:

Defect	With one chapter	With 02 and 02.01
<code>strings.HasSuffix(*rec.Scope, chapter)</code> in the redaction reviewer	correct — one scope, nothing to confuse	02.01 has the suffix 01, so chapter 01's redaction review absorbs sub-chapter artifacts and overstates its own coverage
<code>chapter-\d+</code> in two extraction stages, with <code>continue</code> on a non-match	correct — <code>chapter-01</code> matches	<code>chapter-02.01</code> does not match, and a non-match is skipped silently on a green run

Neither failed loudly. One produced a disclosure-control record that claimed more coverage than it had, and the other produced nothing at all and said nothing about it. That is the class this feature is about: *not* code that crashes on a dotted id, but code that keeps working, keeps returning 200, keeps exiting 0, and is quietly wrong. The remaining open defects listed below are all of that same class, with one exception (`ordinalOf`) that is loudly wrong in a way a reader will misread as a cosmetic label.

And the root of it is not in the code at all. `specs/001-workshop-curriculum-platform/data-model.md:45` types `Chapter.ordinal` as `int`. Every ordinal collision in this document follows from that one line, and no amount of implementation work resolves it — a 02.01 has no `int` ordinal, and giving it one is how 02 and 02.01 come to be the same number. That decision is Phase 0 and it blocks the rest.

Context: Verified Starting State

Measured on **2026-09-05**. Every figure is re-derivable by the command in the right-hand column; none is quoted from a document.

Requirements below are only meaningful against these values.

Fact	Measured value	Re-derive with
Chapter directories present	3 — 01, 02, 02.01. 02.01 is the first sub-chapter	<code>ls workshop/chapters/</code>
Curriculum directories present	1 — chapter-01 only	<code>ls workshop/curriculum/</code>
Records in the passage registry	13,141	<code>count lines in workshop/curriculum/passages.jsonl</code>
Distinct chapter scopes in the registry	1 — 01, holding 1,819 records. 02 and 02.01 hold zero	<code>group the registry by scope</code>
Other scopes in the registry	docs 2,176 , curriculum 2 , and 9,144 records carrying no scope at all	<code>”</code>
Archive state of the new chapters	both archived and verified, not ingested : 02 carries 16 *.part-* members plus a SHA-256 manifest and a .verified marker; 02.01 carries 2. Neither has a transcript	<code>ls workshop/chapters/02 workshop/chapters/02.01</code>
Chapter id accepted by the filesystem resolver	. is already permitted. SafeSlug allowlists a-z A-Z 0-9 - _ . and rejects only / and \	<code>workshop/platform/backend/pkg/curriculum/curriculum.go:205, :209</code>
Chapter directory resolution for a dotted id	already works. ChapterDir tries chapter-<slug> then <slug>	<code>curriculum.go:187</code>

Fact	Measured value	Re-derive with
Route wildcard shape	GET /api/chapters/{chapter} and five siblings. Go's {name} wildcard matches exactly one path segment and has no dot delimiter	workshop/platform/backend/cmd/workshop-server/main.go:547, :817–:819, :841, :843
Client URL construction	encodeURIComponent(slug), which leaves . unescaped — an unreserved character	workshop/platform/frontend/src/app/core/api.ts:107, :122, :155
Chapter list ordering	already byte-lexicographic on the id — sort.Slice(out, func(i, j int) bool { return out[i].ID < out[j].ID })	main.go:2126
Chapter enumeration	listChapters accepts any directory name that is not a dotfile. There is no id grammar and no validator	main.go:2094–2110
Ordinal derivation	ordinalOf("02.01") stops at the first non-digit and returns 2 — the same value as ordinalOf("02")	workshop/platform/backend/internal/api/chapters.go:736
Title derivation	chapterTitle renders both 02 and 02.01 as "Chapter 2"	chapters.go:755–756
The API's own account of that	the derivation.ordinal string — <i>"parsed from the leading digits of the chapter slug"</i> — accurately describes the defect . The endpoint is not lying; it is telling you exactly what it did	chapters.go:140–146
Suggest source manifest	6 rows hardcode chapter-01/..., and the file is compiled in with // go:embed	workshop/platform/backend/pkg/search/suggest-sources.json:10,34,57,77,101,125; pkg/search/catalog.go:70–71

Fact	Measured value	Re-derive with
Leak detector behaviour on a dotted id	HasSourceFilenameShape does not misclassify .01 — its extension list is closed and numeric-only suffixes are not in it	catalog.go:320–354, :368
Front-end ordinal, minimal shape	ordinal: /\^d+ \$/.test(key) ? Number(key) : null — yields null for 02.01	workshop/platform/frontend/src/app/core/models.ts:301
Front-end ordinal, full shape	ordinal: num(raw['ordinal']) — yields 2 for 02.01. The two shapes disagree	models.ts:357
Chapter list consumed flat	the client reads body['chapters'] as an array	api.ts:88
Ingest chapter default	CHAPTER="01"	workshop/scripts/ingest.sh:46
Ingest transcript default	WORKSHOP_TRANSCRIPT_JSON defaults to chapter 01's transcript JSON	ingest.sh:93
Transcript builder defaults	--chapter default="chapter-01", -- title default="Chapter 1"	workshop/pipeline/build_transcript.py:143–144
Calibration input	a frozen absolute path to chapter 01's recording file	workshop/pipeline/calibrate.sh:36
Chapter-01 knowledge builder	2 frozen deep links, 4 frozen chapter_slug literals — and one line that derives correctly from r["scope"]	workshop/curriculum/chapter-01/knowledge/build.py:363, :435; :478, :564, :582,:592; correct at :415
Route manifest chapter substitution	#subst {chapter} 01	workshop/platform/gates/route-manifest.tsv:64
Route manifest literal rows	rows :72–:76 and :85 are written with a literal / api/chapters/01/; rows	”

Fact	Measured value	Re-derive with
A gate that already does it right	:157–:160 are four NOT_BUILT rows carrying the same literal verify-absence-honesty.sh takes a live chapter id from GET /api/chapters rather than hardcoding one	workshop/platform/gates/verify-absence-honesty.sh:127–134

```
# The three commands behind the load-bearing rows
ls workshop/chapters/ workshop/curriculum/
python3 -c "import json,collections;c=collections.Counter(json.loads(l).get('scope'))
for l in open('workshop/curriculum/passages.jsonl'));print(dict(c))"
printf '10\n02.09\n01\n03\n02\n02.10\n02.01.01\n02.01\n' |
LC_ALL=C sort
```

The tree moved DURING drafting, and four rows above are already superseded

Re-measured at the end of the same session that produced the table. **Another agent working in this checkout landed two of the fixes this specification asks for, and ingested a sub-chapter, while it was being written.** That is recorded rather than folded into the table, because a starting state that has quietly been edited to match the present is no longer a starting state.

Row above	As measured	Re-measured, same day	Status
Distinct chapter scopes	1 (01, 1,819 records)	2 — 01 1,819 and 02.01 42	SUPERSEDED
Registry record count	13,141	13,183	SUPERSEDED
Curriculum directories	chapter-01 only	chapter-02.01/ present, untracked	SUPERSEDED
ingest.sh:46 — CHAPTER="01"	frozen default	CHAPTER="" at :60, and a refusal at :109 exiting through the three-valued	FIXED, and the fix is not this feature's

Row above	As measured	Re-measured, same day	Status
ingest.sh:93 — frozen full_ch01 transcript	frozen default	undetermined path derived at :120 as full_ch\$ {_CH_FLAT}, with the dots stripped so a sub-chapter id yields a single- extension filename	FIXED, and the fix is not this feature's

Three consequences, and none of them is “update the table”.

- 1. FR-016 and FR-017 are satisfied in code and unsatisfied in evidence.** The refusal exists; **no test for it appeared.** G-CH-13 and its paired mutation are still owed, and a fix with no paired proof is a fix that reverts the first time somebody finds the refusal inconvenient. Task T017 is annotated accordingly and stays unticked.
- 2. The starting-state table is a dated observation and is now demonstrably one.** Do not quote any figure in it as current — re-run the commands. This document’s own numbers went stale in hours.
- 3. A sub-chapter now holds passages, which changes what the two-scope redaction fixture is testing.** The registry that was hypothetical when Phase 2 was written is real: scopes 01 and 02.01 both exist in curriculum/passages.jsonl today. That makes T013 easier to build and makes T014 more urgent, because the condition under which HasSuffix and == disagree is no longer a fixture — it is the corpus.

Three facts that are load-bearing and easy to misread

- 1. 02 and 02.01 are archived and verified, and they are *not* in the corpus.** Every “0 passages” figure above is a statement about ingestion, not about the material. The recordings exist, hash-verified, in the tree. Nothing in this specification is blocked on acquiring content; it is blocked on the corpus being able to *name* the content it already has.

2. **The filesystem and routing layers are already correct, and that is a measurement, not luck.** SafeSlug permits `.`; `ChapterDir` resolves `chapter-02.01`; the route wildcard is single-segment; `encodeURIComponent` leaves `.` alone; the list comparator is already the right one. **Four of the five places a dotted id has to survive already survive it.** The work is therefore much smaller than “add hierarchy support”, and pretending otherwise would invite a rewrite of code that is already right. The one that is *not* correct is derivation — `ordinalOf` and everything downstream of it.
3. **chapters.go:141 is not a bug in the documentation — it is documentation of the bug.** The endpoint publishes `derivation.ordinal` as “*parsed from the leading digits of the chapter slug*”, which is precisely what it does and precisely why `02.01` collides with `02`. An honest instrument describing a wrong answer is still a wrong answer, and the fix is the derivation, not the string.

Clarifications

Session 2026-09-05

One question is open, it is a **specification** question rather than an implementation one, and it **blocks everything else in this feature**. It is stated first because a reader who skips it will read the requirements below as implementable, and they are not.

- **[NEEDS CLARIFICATION: the ordinal type, and the path key that contradicts it]** Two lines in feature 001’s own artifacts have to be resolved explicitly, and neither can be resolved by writing code:

(a) specs/001-workshop-curriculum-platform/data-model.md:45 types Chapter.ordinal as int. Under that type, `02.01` has no representable ordinal. Every collision in this document — `ordinalOf` returning 2, `chapterTitle` rendering “*Chapter 2*”, the front end’s two disagreeing shapes — is that one line reaching the surface. Three ways out: **(i)** widen `ordinal` to an ordered *path* of integers (this specification’s proposal, `ordinal_path`); **(ii)** keep `int` and declare sub-chapters to have no ordinal, which makes them second-class in every ordering; **(iii)** keep `int` and flatten sub-chapters into the same number space, which renumbers every existing chapter the first time a sub-chapter is inserted. *Scope impact:* (i) changes a wire field and one derivation; (ii) changes nothing and leaves the front end’s `null/2` disagreement as the contracted behaviour; (iii) breaks every existing link.

(b) specs/001-workshop-curriculum-platform/contracts/http-api.md:60 states that the zero-padded ordinal is “not accepted as a path key — one key, one meaning”. The implementation uses **exactly that** as the path key: GET /api/chapters/01 is the live, manifest-declared route (route-manifest.tsv:72), and 01 is the zero-padded ordinal. The contract and the implementation have disagreed since before this feature existed, and the disagreement went unseen because — with one chapter — the slug and the zero-padded ordinal are the same string. **They stop being the same string the moment ordinal is not an int.** Two ways out: amend the contract to say the dotted id is the key (this specification’s proposal — it makes the slug and the id one string, which is the whole identity decision below), or introduce a separate slug and accept two keys for one chapter.

Neither may be inherited silently. Both are recorded in feature 001’s artifacts, both are wrong or contradicted today, and this feature’s entire identity model rests on which way they go. A [BLOCKED: ordinal type] marker is live on the tasks that depend on it.

Everything else the change left unstated was decided, and those decisions are recorded under **Assumptions** and **Resolved Decisions** with their reasoning.

User Scenarios & Testing (*mandatory*)

User Story 1 - A sub-chapter is a chapter (Priority: P1)

A maintainer adds chapters/02.01 and expects the platform to treat it as a chapter: listed, addressable, orderable, ingestible, searchable — without a single line of code that asks whether it is a sub-chapter.

Why this priority: it is the whole feature. Every other story is a consequence.

Independent test: create a chapter directory named with a two-level id, run the pipeline, and confirm it appears in the list, resolves at its own URL, sorts in the right place, and carries no special-cased handling anywhere in its path.

Acceptance scenarios:

1. **Given** chapters/02.01 exists, **When** a client calls the chapter list, **Then** the entry appears with id 02.01, a parent of 02, a depth of 2, and an ordinal path that does not collide with 02.
 2. **Given** the same, **When** a client requests that chapter directly, **Then** it resolves — the route wildcard is single-segment and the id contains no /.
 3. **Given** 01, 02, 02.01, 02.10, 03 and 10 all exist, **When** any of them is listed, globbed or read from disk, **Then** the three orders are identical and 02.01 falls immediately after 02.
 4. **Given** the codebase, **When** it is searched for a branch conditioned on a chapter's depth, **Then** none is found.
-

User Story 2 - Nothing is silently dropped (Priority: P1)

A maintainer runs the extraction, ingestion and authoring stages over a tree containing a sub-chapter, and every stage either processes it or says, by name, that it did not.

Why this priority: also P1, and for a stated reason — the two regex defects already fixed did not fail. They exited 0. A pipeline that drops a chapter silently is worse than one that refuses to run, because the refusal is a fact and the silence is a false one.

Independent test: run every stage against a tree holding a sub-chapter with no content of its own, and assert that the sub-chapter is named in each stage's output — as processed, or as skipped with a reason.

Acceptance scenarios:

1. **Given** a chapter directory whose name does not match the id grammar, **When** any stage enumerates chapters, **Then** it is reported as unclassified with a stated reason and is never omitted from the enumeration.
 2. **Given** a sub-chapter with no transcript, **When** ingestion runs without an explicit transcript argument, **Then** it **refuses**, naming the missing input — it does **not** fall back to another chapter's transcript.
 3. **Given** any stage that filters chapter paths by pattern, **When** the pattern fails to match a directory that exists, **Then** the non-match is reported rather than skipped with continue.
-

User Story 3 - Every chapter-scope comparison names exactly one chapter (Priority: P1)

An operator runs a redaction review for chapter 01 and gets chapter 01's artifacts — not 02.01's, and not 01's plus everything whose id happens to end in 01.

Why this priority: P1, and the reason is disclosure control. The fix has landed (workshop-redact/main.go:616 is now `*rec.Scope != chapter`). What has **not** landed is the test that can see the defect, and a fix without one is a fix that can be reverted by an editor who thinks `HasSuffix` reads more permissively.

Independent test: build a fixture registry holding **two** scopes, 01 and 02.01, run the redaction review for 01, and assert 02.01's artifacts are absent.

Acceptance scenarios:

1. **Given** a registry with scopes 01 and 02.01, **When** the review runs for 01, **Then** exactly the 01 artifacts are covered.
 2. **Given** the same fixture, **When** the comparison is mutated back to `HasSuffix`, **Then** the test goes red.
 3. **Given** a **one**-scope fixture, **When** the same mutation is applied, **Then** the test stays green — which is the measurement that proves a one-scope fixture cannot guard this.
-

User Story 4 - Hierarchy is visible without a new endpoint (Priority: P2)

A client renders a chapter tree, filters to one branch, and asks how deep the corpus goes — using the endpoints it already calls.

Why this priority: the information is needed, and a new route is a new manifest row, a new contract section, a new gate and a new negative assertion. Deriving it costs none of those.

Independent test: fetch the chapter list, build a tree from the hierarchy object on each row without a second request, and confirm the tree matches the directory layout.

Acceptance scenarios:

1. **Given** the chapter list, **When** a client reads a row, **Then** it carries `parent_id`, `depth`, `ancestor_ids`, `child_ids` and `orphaned` — and the array is still **flat**.
 2. **Given** `?under=02`, **When** the list is fetched, **Then** it returns 02's descendants, and the response echoes the filters it applied.
 3. **Given** `?under=99`, **When** the list is fetched, **Then** the response is 200 with an empty array and `under_resolved: false` — **not** a 404, because the request was well-formed and the answer is “*no such branch*”, not “*no such endpoint*”.
 4. **Given** the backend cannot read the chapter tree, **When** the list is fetched, **Then** the response is the existing could-not-determine state **and still echoes the filters**.
-

User Story 5 - An orphan is served, and says so (Priority: P3)

`chapters/02.01` exists and `chapters/02` does not. The platform serves `02.01` and states that its parent is missing.

Why this priority: it is a real state — a sub-chapter can be archived before its parent is, or a parent can be withdrawn — and both of the obvious behaviours are wrong. Hiding the orphan loses content that exists; serving it as a root silently rewrites the hierarchy.

Independent test: remove a parent directory from a fixture tree, list the chapters, and assert the child is present, marked `orphaned: true`, and names the id it could not find.

Acceptance scenarios:

1. **Given** `02.01` with no `02`, **When** the list is fetched, **Then** `02.01` is present, `orphaned` is `true`, and `parent_id` still reads `02` — the derivation is unchanged by the parent's absence.
 2. **Given** the same, **When** `?under=02` is requested, **Then** `02.01` is returned, because `under` filters on the derived ancestry and not on the existence of the ancestor.
-

Edge Cases

- **A directory named 02.2.** Not zero-padded, so byte order puts it *after* 02.10. It must be refused by the validator, not silently sorted wrongly. This is the single input the grammar exists to reject.
- **A directory named 2.** Same class, one level up.
- **A directory named 02.01.01.01.** Depth is unbounded in the grammar. It must be accepted, and the measured maximum depth must be published rather than a limit asserted.
- **A chapter id used as a path key against a route wildcard.** {chapter} is single-segment. An id containing / would silently truncate the match — which is exactly why SafeSlug rejects /, and why the rejected 02/01 design would have needed that rejection removed.
- **A parent with no material of its own.** 02 may be a container. It is still a chapter, is still listed, and its zero passages are a measurement rather than an error.
- **Two chapters whose ids differ only by padding** (02.1 and 02.01). The grammar rejects the first, so this cannot arise — but the validator must be *tested* on it, because “cannot arise” is an assertion about a validator that has not been written yet.
- **A search hit whose scope is a sub-chapter.** Any consumer comparing scopes with HasPrefix, HasSuffix or Contains is wrong for the same reason the redaction reviewer was.
- **A chapter id appearing inside a filename-shaped string.** The leak detector’s extension list is closed and holds no numeric-only entry, so .01 is not read as an extension. Measured, not assumed — and it must stay measured, because the fix for it is one entry in a list.
- **The client’s two chapter shapes disagreeing.** They already do: null from one, 2 from the other. Whatever the ordinal decision is, both shapes must produce the same value or the mismatch must be impossible to express.

Requirements (*mandatory*)

Functional Requirements

Identity

- **FR-001:** A chapter’s canonical identifier **MUST** be its dotted, zero-padded numeric path — 01, 02, 02.01 — and **MUST** match the grammar `^[0-9]{2,}(\.[0-9]{2,})*$`.

- **FR-002:** That one string MUST serve, unchanged and untranslated, as **all five** of: the directory name under chapters/, the URL path segment, the passage registry scope value, the chapter_slug field value, and the suffix in a chapter-<id> curriculum directory name. No layer MAY map it to a different form.
- **FR-003:** Depth MUST be unbounded in the grammar. A limit MAY be measured and published; it MUST NOT be asserted.
- **FR-004:** A directory under chapters/ whose name does not match FR-001 MUST be reported as unclassified, with the reason, in every enumeration that covers it. It MUST NOT be silently skipped, and it MUST NOT be silently accepted.
- **FR-005:** The validator MUST reject a non-zero-padded component (02.2, 2), and this rejection MUST carry a paired mutation proving the check can fail.

Derivation

- **FR-006:** parent_id, depth, ancestor_ids, child_ids and ordinal_path MUST all be **derived from the identifier by string operation** and MUST NOT be stored anywhere.
- **FR-007:** No entity MAY carry a stored parent_id, a stored depth, or a stored child list. A second representation of the hierarchy is forbidden, because two representations of one fact can disagree and the disagreement has no symptom.
- **FR-008:** parent_id MUST be null for a top-level chapter and child_ids MUST be [] for a leaf. Both MUST fall out of the derivation rather than being special-cased.

Ordering

- **FR-009:** The order of chapters MUST be byte-lexicographic on the identifier string, and this MUST be the same order produced by the filesystem, by a shell glob and by the API.
- **FR-010:** A parent MUST sort immediately before its first child. This is a consequence of the zero-padding and of . (0x2E) sorting below 0–9 (0x30–0x39); it MUST NOT be implemented as a separate rule.
- **FR-011:** The existing comparator at main.go:2126 MUST NOT be changed. It is already correct, and a gate MUST assert the ordering rather than the code so that a future rewrite is caught.

The scope-comparison rule

- **FR-012:** Every comparison of a chapter scope against a chapter id MUST be **equality**. Use of HasSuffix, HasPrefix or Contains on a chapter scope is a defect.

- **FR-013:** The equality comparison at `workshop-redact/main.go:616` MUST be covered by a test whose fixture registry holds **at least two** scopes, one of which is a sub-chapter of the other. A single-scope fixture MUST NOT be accepted as coverage, and the paired proof MUST demonstrate that a single-scope fixture stays green under the `HasSuffix` mutation.

Pattern widening

- **FR-014:** Every pattern matching a chapter path MUST accept a dotted id. The two already widened — `_TRANSCRIPT_PATH_RE` at `pipeline/extract/meeting_notes.py:109` and `_ELIGIBLE_SOURCE` at `pipeline/extract/author.py:159` — MUST retain paired mutations asserting **both** directions: a dotted id now matches, **and** the previously-matching flat form still matches. A widening that quietly stopped matching `chapter-01` would be worse than the defect it fixed.
- **FR-015:** A pattern non-match on a directory that exists MUST be reported. `continue` on a non-match is forbidden wherever the set being iterated is the set of real chapters.

Silent-content defects

- **FR-016:** `workshop/scripts/ingest.sh` MUST NOT default its transcript input to any specific chapter's transcript. Omitting the transcript export while naming a chapter MUST **refuse**, naming the missing input — it MUST NOT ingest a different chapter's content under the requested scope.
- **FR-017:** `ingest.sh`'s chapter default MUST be removed or MUST be reported on every run. A default that names a real chapter is indistinguishable, in the output, from an explicit choice.
- **FR-018:** `workshop/pipeline/build_transcript.py:143–144` MUST NOT carry frozen `chapter-01 / Chapter 1` defaults.
- **FR-019:** `workshop/pipeline/calibrate.sh:36` MUST resolve its recording input from a named chapter rather than from a frozen absolute path.
- **FR-020:** `workshop/curriculum/chapter-01/knowledge/build.py` MUST derive every chapter reference from the record it is processing, as `:415` already does. The frozen deep links at `:363` and `:435` and the frozen `chapter_slug` literals at `:478`, `:564`, `:582` and `:592` MUST be derived the same way.
- **FR-021:** The suggest source manifest MUST NOT hardcode a chapter directory. The six `chapter-01/... rows` at `suggest-sources.json:10,34,57,77,101,125` MUST be replaced by a

derivation over the chapters that exist, so that a new chapter reaches `/api/suggest` without a rebuild of an embedded file.

Visible-label defects

- **FR-022:** `ordinalOf` MUST NOT return a value that collides between a parent and its child. Its replacement MUST be an ordered path, and the resolution of the `ordinal` type question above governs what that path is called on the wire.
- **FR-023:** `chapterTitle` MUST NOT render two distinct chapters with the same title.
- **FR-024:** `derivation.ordinal` MUST be rewritten when the derivation is, and MUST continue to describe what the code actually does. It is the endpoint's honesty mechanism and it currently works; it MUST NOT be left describing a derivation that has been replaced.
- **FR-025:** The front end's two chapter shapes MUST produce the same `ordinal` value for the same chapter, or the wire format MUST make the disagreement unrepresentable. `models.ts:301` and `models.ts:357` currently yield `null` and `2` for `02.01`.

API

- **FR-026: No new route MAY be added by this feature.** The information is carried by the two existing chapter endpoints.
- **FR-027:** `GET /api/chapters` MUST carry a hierarchy object on every row, and the top-level chapters value MUST remain a **flat array**. Nesting it would break `api.ts:88`, which reads it flat.
- **FR-028:** `GET /api/chapters` MUST accept `under`, `depth` and `include_self`, and MUST echo the filters it applied on **every** response status, including `could-not-determine`. A client that cannot see which filters were applied cannot distinguish an empty branch from an ignored parameter.
- **FR-029:** An `under` value naming a chapter that does not exist MUST return `200` with an empty array and `under_resolved: false`. It MUST NOT return `404`: the request was well-formed and the route exists.
- **FR-030:** `GET /api/chapters/{chapter}` MUST carry a hierarchy object with resolvable parent and child hrefs, MUST replace `ordinal` with `ordinal_path`, MUST fix `title`, and MUST rewrite `derivation.ordinal`.
- **FR-031:** An orphaned chapter MUST be **served**, with `orphaned: true` and the missing ancestor named. It MUST NOT be hidden and MUST NOT be silently promoted to a root.

No special-casing

- **FR-032:** There MUST be **no sub-chapter-specific code path anywhere**. A branch conditioned on a chapter's depth — `if depth > 1` — is a defect, because it means the general case was not built.

Governance

- **FR-033:** Every route touched by this feature MUST have its route-manifest row updated, and the manifest MUST NOT continue to encode 01 as the only substitutable chapter.
- **FR-034:** Every gate this feature adds MUST be three-valued — 0 clean, 1 a real finding, 2 could not determine — and MUST carry a paired mutation proof.
- **FR-035:** Every gate that needs a chapter id MUST take it from the live tree, as `verify-absence-honesty.sh:127–134` already does, rather than hardcoding one.

Key Entities

- **Chapter:** unchanged in kind, changed in identity. Its identifier becomes the dotted numeric path, which is simultaneously its directory name, its URL segment, its registry scope, its `chapter_slug` and its chapter- suffix. It gains no stored field.
- **Chapter Hierarchy: not an entity** — a derived view. `parent_id`, `depth`, `ancestor_ids`, `child_ids` and `ordinal_path` are computed from the identifier string on demand. It is named here only because the API returns it and a thing the API returns needs a defined shape.
- **Passage:** unchanged. Its `scope` field already holds a chapter id and already holds it as an opaque string; this feature changes what strings appear there, not the field.

Success Criteria (*mandatory*)

Measurable Outcomes

Every criterion names the method that measures it, and every criterion guarding a silent failure names the mutation that must turn it red.

Identity and ordering

- **SC-001:** 100% of chapter directories are either accepted by the grammar or reported unclassified with a reason. **Measured by:**

enumerating chapters/, partitioning into accepted and reported, and asserting the two counts sum to the directory count. Paired mutation: make the enumerator continue on a non-match — the check must go red.

- **SC-002:** The validator rejects 02.2, 2, 2.1, 02., .01 and the empty string, and accepts 01, 02, 02.01, 02.01.01 and 100. **Measured by:** a table-driven test over exactly those inputs. Paired mutation: relax the grammar to [0-9]+ — the rejections must fail.
- **SC-003:** API order, glob order and filesystem byte order are **identical** over a fixture tree holding 01 02 02.01 02.01.01 02.09 02.10 03 10. **Measured by:** capturing all three and comparing them element by element; any difference fails. Paired mutation: sort by `ordinal_path` numerically instead of by the id string — the check must go red on 02.09 versus 02.10.
- **SC-004:** A parent sorts immediately before its first child in 100% of parent/child pairs present. **Measured by:** for each parent, asserting the next element in the ordering is its first child whenever it has one.

No silent loss

- **SC-005:** Every pipeline stage that enumerates chapters names every chapter directory in its output — as processed or as skipped-with-reason. **Measured by:** running each stage against a fixture tree containing one sub-chapter and one deliberately malformed directory, then asserting both names appear in the output. Paired mutation: restore continue on non-match in one stage — the check must go red.
- **SC-006:** Both widened patterns match both forms. **Measured by:** asserting `_TRANSCRIPT_PATH_RE` and `_ELIGIBLE_SOURCE` each match a chapter-01 path **and** a chapter-02.01 path. Paired mutations, both required: (a) revert to `chapter-\d+` — the dotted assertion must fail; (b) change to `chapter-\d+\. \d+` — the flat assertion must fail.
- **SC-007:** Ingestion never ingests one chapter's transcript under another chapter's scope. **Measured by:** invoking `ingest.sh` with a chapter and no transcript export and asserting a non-zero exit naming the missing input, with **zero** records written. Paired mutation: restore the `full_ch01` default — the check must go red.

Scope comparison

- **SC-008:** The redaction reviewer's coverage for chapter 01 excludes 02.01's artifacts. **Measured by:** a **two-scope** fixture registry, asserting the covered artifact set equals the 01 set exactly. Paired mutation: restore `strings.HasSuffix` — the check must go red.

- **SC-009:** A **one-scope** fixture is proven insufficient. **Measured by:** running the same `HasSuffix` mutation against a single-scope fixture and asserting the test **stays green**. This criterion measures the test's own blindness, and it is the reason SC-008 is specified with a fixture shape rather than only an assertion.
- **SC-010:** 0 chapter-scope comparisons in the tree use `HasSuffix`, `HasPrefix` or `Contains`. **Measured by:** a gate that enumerates comparisons against a chapter-scope value and asserts the operator is equality. Paired mutation: introduce one — the check must go red.

Derivation

- **SC-011:** 0 hierarchy fields are stored. **Measured by:** inspecting the registry schema, the derived database and every serialised artifact for a persisted `parent_id`, `depth` or `child` list; the count must be 0. Paired mutation: persist one — the check must go red.
- **SC-012:** For 100% of chapters, every derived field recomputes identically from the identifier alone. **Measured by:** recomputing the whole hierarchy from the id list with no filesystem access and comparing to what the API returns.
- **SC-013:** `ordinalOf` produces no collision. **Measured by:** computing the ordinal representation for every chapter present plus a synthetic set of at least eight ids and asserting the values are pairwise distinct. Paired mutation: restore leading-digit parsing — the check must go red on the 02 / 02.01 pair.
- **SC-014:** The front end's two chapter shapes agree. **Measured by:** constructing both from the same payload for every chapter present and asserting field-by-field equality of the ordinal representation. Paired mutation: restore the `/^\d+$/` test in the minimal shape — the check must go red on 02.01.

API

- **SC-015:** 0 new routes are added. **Measured by:** diffing the route manifest's route set before and after; the added-route count must be 0.
- **SC-016:** The `chapters` value is a flat array in 100% of responses. **Measured by:** asserting the JSON type and asserting no element carries a nested chapter array.
- **SC-017:** The `filters echo` is present on **every** status, including `could-not-determine`. **Measured by:** driving the endpoint into each of its states with `filters` set and asserting the `echo` in each. Paired mutation: omit the `echo` on the unavailable path — the check must

go red. This is the criterion that keeps a degraded backend from looking like an empty branch.

- **SC-018:** An unknown under returns 200 with `under_resolved: false`, never 404. **Measured by:** probing with a chapter id that does not exist and asserting the status and the field.
- **SC-019:** An orphaned sub-chapter is served with `orphaned: true` and names its missing ancestor. **Measured by:** a fixture tree with the parent removed. Paired mutation: filter orphans out of the list — the check must go red.

No special-casing

- **SC-020:** 0 code paths branch on chapter depth. **Measured by:** a gate that searches the backend, front end and pipeline for a conditional on a depth or level value derived from a chapter id, and asserts none exists. Any hit is reported with its location rather than counted.
- **SC-021:** The maximum depth present is **measured and published**, and no depth limit is asserted anywhere. **Measured by:** computing the maximum over the chapters present and asserting no constant bounding depth exists in the codebase.

Backward compatibility

- **SC-022:** 01 is valid under the grammar and its ordinal representation is unchanged in value for every chapter that exists today. **Measured by:** computing the representation for every present chapter before and after and asserting equality. This is the criterion that makes the change non-breaking for the corpus as it stands.
- **SC-023:** Every deep link that resolves today still resolves after the change. **Measured by:** enumerating the deep links in the built artifacts, resolving each before and after, and comparing the outcome sets.

Governance

- **SC-024:** 100% of gates added by this feature are three-valued and carry a paired mutation. **Measured by:** driving each into a could-not-determine condition and observing exit 2, then running its prover.
- **SC-025:** 0 gates hardcode a chapter id. **Measured by:** a check over the gate scripts asserting each obtains its chapter from the live tree or from an argument.

Assumptions

- **The dotted id is a path, not a decimal.** 02.10 sorts after 02.09 because both are zero-padded strings compared byte by byte, not because .10 > .09 numerically. Reading the id as a decimal is the single most likely misreading and it produces the wrong order at exactly ten children.
- **Zero-padding is two digits minimum, and more is allowed.** `^[0-9]{2,}` accepts 100. A hundred-chapter corpus is not planned; a grammar that forbids it for no reason is a limit asserted rather than measured.
- **The four already-correct layers stay correct by accident of good design, and are now asserted.** SafeSlug, ChapterDir, the route wildcard and encodeURIComponent all handle . today. Each is one small edit away from not doing so, so each acquires an assertion rather than a comment.
- **02 may legitimately have no material of its own.** A parent is a container as well as a chapter. Zero passages under 02 is a measurement to publish, not an error to raise.
- **The corpus grows; the identity does not change.** 02 and 02.01 are archived and archived and not ingested today. Ingesting them is separate work that this feature makes *possible*; this feature is complete when a sub-chapter can go through the pipeline unchanged, not when a particular sub-chapter has.
- **No renumbering, ever.** Existing chapter ids are immutable. Any option that renumbers 01 is rejected on that ground alone, because every deep link, every registry scope and every archived artifact resolves through it.
- **Governance is binding.** Three-valued checks, paired mutation proofs, no server-side CI, the content boundary in both directions, and the continuation document updated alongside non-trivial changes.

Dependencies

- **The resolution of the ordinal type question and the path-key contradiction** (Clarifications above). Both are recorded in feature 001's artifacts, both must be amended explicitly, and this feature's wire format depends on which way each goes. Nothing downstream of Phase 0 is implementable until they are settled.
- **The existing passage registry and its scope field**, which already stores a chapter id as an opaque string and needs no change to hold a dotted one.

- **The existing identity library**, consumed from `submodules/passage`. Chapter ids are **not** minted identifiers and this feature does not make them one — see Resolved Decisions D3.
- **The route manifest and the server-unity verifier**, which must be updated in the same change as any endpoint behaviour, and whose `#subst {chapter} 01` line is itself a chapter-01 assumption.
- **Feature 002’s knowledge layer**, which consumes `chapter_slug` and derives deep links. Its own builder is one of the artifacts listed under FR-020.

Out of Scope

- **Ingesting chapters 02 and 02.01.** They are archived and verified; running them through the pipeline is the work this feature enables and is not this feature.
- **Any new endpoint.** FR-026 states it as a requirement rather than an exclusion, and it is repeated here because “add `/api/chapters/{chapter}/children`” is the obvious first idea.
- **A nested chapter tree on the wire.** The array stays flat; nesting is a client concern and a breaking change.
- **A depth limit.** Measured and published, never asserted.
- **Renaming or renumbering any existing chapter.**
- **Re-recording, re-transcribing or editing source material.**
- **A hierarchy for anything other than chapters.** Knowledge areas are flat by feature 002’s own decision and nothing here changes that.

Resolved Decisions

Questions settled here rather than left open, each recording what was chosen, why it is the safe choice, and what would reopen it.

D1 — The canonical id is the dotted, zero-padded numeric path

The question: how is a sub-chapter identified?

Decision: `^[0-9]{2,}(\.[0-9]{2,})*$`. One string, five roles — directory name, URL segment, registry scope, `chapter_slug`, and the suffix of `chapter-<id>` — with **no translation at any boundary**.

Why this is the safe choice: it is the form the tree already uses (chapters/02.01 exists), and **four of the five layers it has to survive already survive it**, measured rather than assumed: SafeSlug permits . (curriculum.go:205, :209), ChapterDir already resolves chapter-02.01 (:187), Go's {chapter} wildcard is single-segment with no dot delimiter, and encodeURIComponent leaves . unescaped. The fifth — the search catalogue's leak detector — was checked and does not misclassify .01, because its extension list is closed and holds no numeric-only entry (catalog.go:320–354). A scheme requiring **zero** changes to the identity, routing and filesystem layers is not merely convenient; it is the one with the smallest surface on which a silent defect can appear.

What would reopen it: a requirement for a non-numeric chapter id, which would break the ordering guarantee in D2 and would need a different one.

D2 — Ordering is byte-lexicographic on the id, and that is a consequence, not a rule

The question: how do chapters sort, and where does a child go relative to its parent?

Decision: **byte-lexicographic on the identifier string**. Given zero-padding, this produces 01 02 02.01 02.01.01 02.09 02.10 03 10 — verified by running it, not reasoned about. A parent sorts immediately before its first child because . is **0x2E** and the digits 0–9 are **0x30–0x39**, so the child's . beats any sibling's next character.

Why this is the safe choice: the comparator that implements it **already exists and is already correct** (main.go:2126), so the safest action on the sorting code is to change nothing. What is missing is the *guarantee*: nothing today asserts that the three orders — API, glob, filesystem — agree, and nothing stops a future rewrite from “improving” the comparator into a numeric one that puts 02.10 before 02.09. The gate goes on the ordering; the code stays.

The load-bearing consequence: **the validator is what needs the gate, not the sort**. The order is correct *only because every component is zero-padded*. 02.2 would sort after 02.10 and be wrong in a way no sorting test with well-formed input can see. The grammar is therefore the mechanism that makes the ordering true, and rejecting 02.2 is the assertion that carries it.

What would reopen it: variable-width components, which would force a parsed comparator and give up the property that filesystem order and API order are the same order by construction.

D3 — Hierarchy is derived, never stored

The question: store `parent_id` on each chapter, or derive it from the id?

Decision: **derive everything** — `parent_id`, `depth`, `ancestor_ids`, `child_ids`, `ordinal_path` — by string operation on the identifier. Nothing is persisted.

Why this is the safe choice: a stored `parent_id` is a **second source of truth about structure**, and the two can disagree. A chapter whose id says `02.01` and whose stored parent says `03` is not a detectable error state — it is two facts, both readable, both plausible, with no symptom until something navigates. Derivation makes the disagreement unrepresentable. This is the same reasoning feature 002 applied when it refused stored reverse edges, and the same reasoning feature 001 applied when it kept chapter sections derived.

The cost, stated: the id becomes structurally meaningful, so a chapter cannot be re-parented without being renamed. That is accepted: re-parenting a chapter changes what it is, and a rename is a visible operation where a stored-field edit is not.

What would reopen it: a requirement to re-parent chapters without renaming them.

D4 — The identity scheme's two rejected alternatives

The question: two other schemes were considered and both are recorded, because each looks better than it is.

Rejected — `chapters/02/01` as path segments. It reads more naturally as a hierarchy. It requires `/` inside the chapter slug, and `SafeSlug` (`curriculum.go:209`) exists **precisely to reject** `/` — it is a path-traversal boundary, and weakening it to accommodate a naming preference trades a security property for aesthetics. It also breaks the route wildcard: Go's `{chapter}` matches a single segment, so `GET /api/chapters/02/01` would not match the registered pattern at all.

Rejected — a flat id with a stored `parent_id`. It is the conventional relational answer and it creates exactly the second source of truth D3 exists to prevent. It also loses the ordering property: with an opaque

flat id, sort order has to be reconstructed by walking the parent links, and filesystem order stops matching API order by construction rather than agreeing with it.

What would reopen either: nothing currently foreseen. Both are recorded so that the next reader does not re-propose them as improvements.

D5 — No new routes, and the array stays flat

The question: hierarchy has to reach the client. A new endpoint, a nested array, or fields on the existing rows?

Decision: **fields on the existing rows**, plus three query parameters. GET /api/chapters gains a hierarchy object per row; GET /api/chapters/{chapter} gains one with parent and child hrefs. The top-level array stays flat.

Why this is the safe choice: nesting the array is a breaking change to a live consumer — `api.ts:88` reads `body['chapters']` as a flat array — and it would force every client to walk a tree to find a chapter it can currently index. A new route is not free either: the route manifest, the contract section, the server-unity verifier row and a negative assertion all follow it, and the information is fully derivable from data the endpoint already returns.

The one shape that follows from it: an unknown under value must be 200 with `under_resolved: false`, because the route exists and the request was well-formed. A 404 there would say “*no such endpoint*” about a request whose real answer is “*no such branch*” — the same conflation between *absent* and *not-found* that the platform’s four-outcome resolver exists to prevent.

What would reopen it: a client requirement that cannot be met by a flat array plus derived ancestry, which has not been demonstrated.